

Pre-Taller 5: Línea de Ensamblaje Automotriz

Malak Sanchez

Monitorías de Sistemas Operativos

16 de mayo de 2026

Introducción

Este ejercicio está diseñado para aplicar y dominar las variables de condición (`pthread_cond_t`) junto con mutex. En escenarios concurrentes, los hilos requieren pausar su ejecución en espera de que un estado semántico se cumpla.

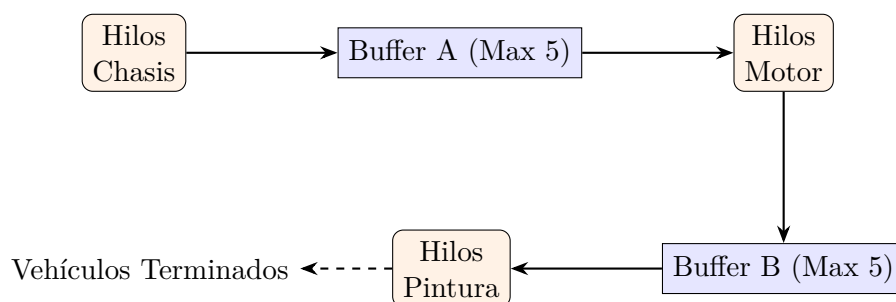
Problema: Productor y Consumidor en Etapas

Diseñe un simulador de una fábrica de automóviles dividida en tres estaciones: **Producción de Chasis**, **Instalación de Motor y Pintura** y **Acabado**.

Cada estación es operada por uno o múltiples hilos. El flujo es un ensamble en cadena, donde:

- Los trabajadores de Chasis alimentan un almacén intermedio *A*.
- Los trabajadores de Motores toman piezas de *A*, acoplan el motor, y mueven el vehículo al almacén intermedio *B*.
- Los trabajadores de Pintura toman del almacén *B*, terminan el vehículo y lo cuentan como un producto finalizado.

Ambos almacenes intermedios (*buffers*) tienen una capacidad limitada. Si un almacén está lleno, los productores anteriores deben **quedar bloqueados temporalmente** hasta que haya espacio. Si un almacén está vacío, los consumidores deben esperar hasta que lleguen nuevas unidades.



Ejemplo de ejecución esperado:

```
$ ./linea_ensamblaje 20
[Sistema] Meta de produccion: 20 vehiculos.
[Chasis 1] Fabrica chasis (Buffer A: 1/5)
[Chasis 2] Fabrica chasis (Buffer A: 2/5)
[Motor 1] Toma chasis, acopla motor (Buffer A: 1/5, Buffer B:
1/5)
[Pintura 1] Pinta vehiculo. !Vehiculo terminado! (Total: 1)
...
[Sistema] Produccion de 20 vehiculos completada con exito.
```

Requerimientos Técnicos

1. En la implementación es de carácter obligatorio utilizar la combinación cíclica de `pthread_mutex_lock`, bucles `while` y `pthread_cond_wait` para la supervisión del inventario en los *buffers*.
2. El programa debe terminar su ejecución de forma completamente limpia una vez se ha alcanzado la meta de vehículos a producir. Para esto, debe emitir las señales de variables de condición (**broadcast**) necesarias para despertar a los hilos en espera y notificarles que el programa ha terminado.

Criterios de Evaluación

- **Correctitud Concurrente:** Uso óptimo de las variables de condición. Ausencia comprobada de condiciones donde hilos consumidores se queden esperando para siempre al final de la jornada.
- **Manejo de Limites:** Garantizar de forma estricta que nunca se rebosan ni se leen *buffers* vacíos.